

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

**Лабораторна робота №3
з дисципліни
«Алгоритми і структури даних»**

Виконала:
студентка групи ІМ-41
Куц Анна Василівна
номер у списку групи: 11

Перевірів:
Сергієнко А.М.

Київ 2025

Постановка задачі

1. Представити у програмі напрямлений і ненаправлений графи з заданими параметрами:
 - кількість вершин n ;
 - розміщення вершин;
 - матриця суміжності A .
2. Створити програму для формування зображення направленого і ненаправленого графів у графічному вікні.

Згадані вище параметри графа задаються на основі чотиризначного номера варіанту $n_1n_2n_3n_4$, де n_1n_2 це десяткові цифри номера групи, а n_3n_4 — десяткові цифри номера варіанту, який був у студента для двох попередніх робіт (див. таблицю з поточними оцінками з АСД, надану викладачем на початку поточного семестру).

Кількість вершин n дорівнює $10 + n_3$.

Розміщення вершин:

- колом при $n_4 = 0, 1$;
- квадратом (прямокутником) при $n_4 = 2, 3$;
- трикутником при $n_4 = 4, 5$;
- колом з вершиною в центрі при $n_4 = 6, 7$;
- квадратом (прямокутником) з вершиною в центрі при $n_4 = 8, 9$.

--

Матриця суміжності A_{dir} напрямленого графа за варіантом формується таким чином:

- 1) встановлюється параметр (seed) генератора випадкових чисел, рівний номеру варіанту $n_1n_2n_3n_4$ — детальніше див. с. 12;
- 2) матриця розміром $n \cdot n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$;
- 3) обчислюється коефіцієнт $k = 1.0 - n_3 \cdot 0.02 - n_4 \cdot 0.005 - 0.25$;
- 4) кожен елемент матриці множиться на коефіцієнт k ;
- 5) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0.

При проєктуванні програм **слід врахувати наступне**:

- 1) мова програмування обирається студентом самостійно;
- 2) графічне зображення графа має формуватися на основі графічних примітивів з графічної бібліотеки (таких як еліпс, пряма, дуга, текст тощо);
- 3) використання готових бібліотек для роботи з графами не дозволяється;
- 4) вивід графа має бути реалізований універсальним чином: вершини і ребра мають виводитися в циклі, а не окремими командами для кожного графічного елемента;
- 5) типи та структури даних для внутрішнього представлення графа у програмі слід вибрати самостійно;
- 6) матриці суміжності графів можна виводити в графічне вікно або консоль — на розсуд студента;
- 7) матриці суміжності мають виводитися як матриці: в квадратному вигляді, з 1 та 0.

Варіант №4111

$$n_1 n_2 n_3 n_4 = 4111$$

$$k = 1.0 - n_3 * 0.02 - n_4 * 0.005 - 0.25 = 1.0 - 0.02 - 0.005 - 0.25 = 1.0 - 0.275 = 0.725$$

Розміщення колом. Кількість вершин = $10 + n_3 = 11$

Оскільки, в обраній мові JavaScript не існує `srand(seed)`, можна використати алгоритм генератора Парка Міллера для отримання псевдовипадкового числа:

$$X_{k+1} = a \cdot X_k \bmod m,$$

Алгоритм було створено мовою програмування JavaScript з використанням математичних бібліотек та вбудовану графічну бібліотеку `<canvas>`, для відображення використовувався HTML та CSS.

Текст програми

index.html

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Graph</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<h1>Variant 4111🎀</h1>

<div>
  <button id="btnDirected">Directed</button>
  <button id="btnUndirected">Undirected</button>
</div>
<canvas id="canvas" width="800" height="800"></canvas>

<script src="script.js"></script>
</body>
</html>
```

styles.css

```
body {
  font-family: "Times New Roman";
  display: flex;
  flex-direction: column;
  align-items: center;
  background: #ffe6f2;
}

canvas {
  border: 1px solid #ccc;
  background: #fff;
  margin-top: 1rem;
}
```

```

}

h1 {
  color: hotpink;
}

button {
  margin: 0.5rem;
  padding: 0.5rem 1rem;
  font-size: 16px;
  cursor: pointer;
  background: hotpink;
  border: none;
  border-radius: 10px;
  color: white;
  font-weight: bold;
  transition: background 0.2s, transform 0.2s;
}

button:hover {
  background: #ff69b4;
  transform: scale(1.05);
}

```

script.js

```

document.addEventListener("DOMContentLoaded", () => {
  const canvas = document.getElementById("canvas");
  const ctx = canvas.getContext("2d");

  //parametres
  const seed = 4111;
  const n3 = 1;
  const n4 = 1;
  const n = 10 + n3;
  const k = 1.0 - n3 * 0.02 - n4 * 0.005 - 0.25; // 0.725

  //Park-Miller's algorythm
  function genRand(seed) {
    const MOD = 2147483647;
    let val = seed % MOD;
    return function () {
      val = (val * 16807) % MOD;
      return (val - 1) / MOD;
    };
  }

  const rand = genRand(seed);

  //matrix generation

```

```

function genDirMatrix(k) {
    const raw = Array.from({length: n}, () =>
        Array.from({length: n}, () => rand() * 2)
    );
    const dir = raw.map(row => row.map(v => (v * k >= 1 ? 1 :
0)))));
    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            if (dir[i][j] && dir[j][i]) {
                if (rand() < 0.5) dir[i][j] = 0;
                else dir[j][i] = 0;
            }
        }
    }
    return dir;
}

function genUndirMatrix(dir) {
    const undir = Array.from({length: n}, () =>
Array(n).fill(0));
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            if (dir[i][j] || dir[j][i]) undir[i][j] =
undir[j][i] = 1;
        }
    }
    return undir;
}

//console print
function printMatrix(matrix, title) {
    console.log(`\n${title}:`);
    matrix.forEach(row => console.log(row.join(" ")));
}

const w = canvas.width;
const h = canvas.height;
const RAD = 20;
const centerX = w / 2;
const centerY = h / 2;
const radius = 280;

const positions = Array.from({length: n}, (_, i) => {
    const angle = (2 * Math.PI * i) / n - Math.PI / 2;
    return {
        x: centerX + radius * Math.cos(angle),
        y: centerY + radius * Math.sin(angle)
    };
});

function distanceToLine(p1, p2, p) {
    const A = p.x - p1.x, B = p.y - p1.y;
    const C = p2.x - p1.x, D = p2.y - p1.y;

```

```

const dot = A * C + B * D, len2 = C * C + D * D;
const param = dot / len2;
let xx, yy;
if (param < 0) {
    xx = p1.x;
    yy = p1.y;
} else if (param > 1) {
    xx = p2.x;
    yy = p2.y;
} else {
    xx = p1.x + param * C;
    yy = p1.y + param * D;
}
const dx = p.x - xx, dy = p.y - yy;
return Math.hypot(dx, dy);
}

function drawArrow(p1, p2, cp = null) {
    let angle;
    if (cp) {
        const t = 0.95;
        const x = (1 - t) ** 2 * p1.x + 2 * (1 - t) * t * cp.x
+ t ** 2 * p2.x;
        const y = (1 - t) ** 2 * p1.y + 2 * (1 - t) * t * cp.y
+ t ** 2 * p2.y;
        const dx = 2 * (1 - t) * (cp.x - p1.x) + 2 * t * (p2.x
- cp.x);
        const dy = 2 * (1 - t) * (cp.y - p1.y) + 2 * t * (p2.y
- cp.y);
        angle = Math.atan2(dy, dx);
        p2 = {x, y};
    } else {
        const dx = p2.x - p1.x, dy = p2.y - p1.y;
        angle = Math.atan2(dy, dx);
        p2 = {
            x: p2.x - RAD * Math.cos(angle),
            y: p2.y - RAD * Math.sin(angle)
        };
    }

    ctx.beginPath();
    ctx.moveTo(p2.x, p2.y);
    ctx.lineTo(
        p2.x - 10 * Math.cos(angle - Math.PI / 8),
        p2.y - 10 * Math.sin(angle - Math.PI / 8)
    );
    ctx.lineTo(
        p2.x - 10 * Math.cos(angle + Math.PI / 8),
        p2.y - 10 * Math.sin(angle + Math.PI / 8)
    );
    ctx.closePath();
    ctx.fill();
}

```

```

function drawSelfLoop(nodeX, nodeY, directed) {
    const arcR = RAD * 0.75;
    const offset = RAD + 10;

    const dx = nodeX - centerX;
    const dy = nodeY - centerY;
    let theta = Math.atan2(dy, dx) * 180 / Math.PI;
    if (theta < 0) theta += 360;
    let cx, cy, start, end;
    if (theta >= 315 || theta < 45) {
        // right
        cx = nodeX + offset;
        cy = nodeY;
        start = -135;
        end = 135;
    } else if (theta >= 45 && theta < 135) {
        // down
        cx = nodeX;
        cy = nodeY + offset;
        start = 225;
        end = 135;
    } else if (theta >= 135 && theta < 225) {
        // left
        cx = nodeX - offset;
        cy = nodeY;
        start = Math.PI / 4;
        end = -45;
    } else {
        // up
        cx = nodeX;
        cy = nodeY - offset;
        start = Math.PI / 4;
        end = -Math.PI / 4;
    }
    //into radians
    const s = start * Math.PI / 180;
    const e = end * Math.PI / 180;

    ctx.beginPath();
    ctx.arc(cx, cy, arcR, s, end, false);
    ctx.stroke();

    if (!directed) return;

    const ax = cx + arcR * Math.cos(e);
    const ay = cy + arcR * Math.sin(e);
    const arrowAngle = Math.atan2(nodeY - ay, nodeX - ax);
    const L = 0.55 * arcR;

    ctx.beginPath();
    ctx.moveTo(ax, ay);
    ctx.lineTo(

```



```

        ax = L * Math.cos(arrowAngle - Math.PI / 6),
        ay = L * Math.sin(arrowAngle - Math.PI / 6)
    );
    ctx.lineTo(
        ax = L * Math.cos(arrowAngle + Math.PI / 6),
        ay = L * Math.sin(arrowAngle + Math.PI / 6)
    );
    ctx.closePath();
    ctx.fill();
}

//draw
function drawGraph(matrix, directed) {
    ctx.clearRect(0, 0, w, h);
    ctx.strokeStyle = "#333";
    ctx.fillStyle = "#000";

    // edges
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            if (!matrix[i][j]) continue;
            if (!directed && j < i) continue;

            if (i === j) {
                drawSelfLoop(positions[i].x, positions[i].y,
directed, i);
                continue;
            }

            const p1 = positions[i], p2 = positions[j];
            let curved = false, cp = null;
            for (let k2 = 0; k2 < n; k2++) {
                if (k2 === i || k2 === j) continue;
                if (distanceToLine(p1, p2, positions[k2]) <
25) {
                    curved = true;
                    const mid = {x: (p1.x + p2.x) / 2, y:
(p1.y + p2.y) / 2};
                    const perp = {x: -(p2.y - p1.y), y: p2.x -
p1.x};
                    const len = Math.hypot(perp.x, perp.y);
                    const dirSign = i < j ? 1 : -1;
                    cp = {
                        x: mid.x + dirSign * (perp.x / len) *
90,
                        y: mid.y + dirSign * (perp.y / len) *
90
                    };
                }
            }
            break;
        }
    }
}

```

```

        ctx.beginPath();
        ctx.moveTo(p1.x, p1.y);
        if (curved) ctx.quadraticCurveTo(cp.x, cp.y, p2.x,
p2.y);

        else ctx.lineTo(p2.x, p2.y);
        ctx.stroke();

        if (directed) drawArrow(p1, p2, curved ? cp :
null);
    }
}

// vertices
for (let i = 0; i < n; i++) {
    ctx.beginPath();
    ctx.fillStyle = "#fff";
    ctx.arc(positions[i].x, positions[i].y, RAD, 0, 2 *
Math.PI);

    ctx.fill();
    ctx.stroke();
    ctx.fillStyle = "#000";
    ctx.font = "14px Times New Roman";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(i + 1, positions[i].x, positions[i].y);
}

}

//initialize
const dirMatrix = genDirMatrix(k);
const undirMatrix = genUndirMatrix(dirMatrix);

document.getElementById("btnDirected").onclick = () => {
    console.clear();
    printMatrix(dirMatrix, "Directed Matrix (Adir)");
    drawGraph(dirMatrix, true);
};

document.getElementById("btnUndirected").onclick = () => {
    console.clear();
    printMatrix(undirMatrix, "Undirected Matrix (Aundir)");
    drawGraph(undirMatrix, false);
};
});

```

Результати тестування програм

Directed Matrix (A_{dir}):

0	1	0	0	0	0	0	0	1	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	1	1	0	0
0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
0	1	0	1	0	1	0	1	1	0	0	0
0	0	1	1	1	0	1	0	0	0	0	0

Напрявлена матриця суміжності

Undirected Matrix (A_{undir}):

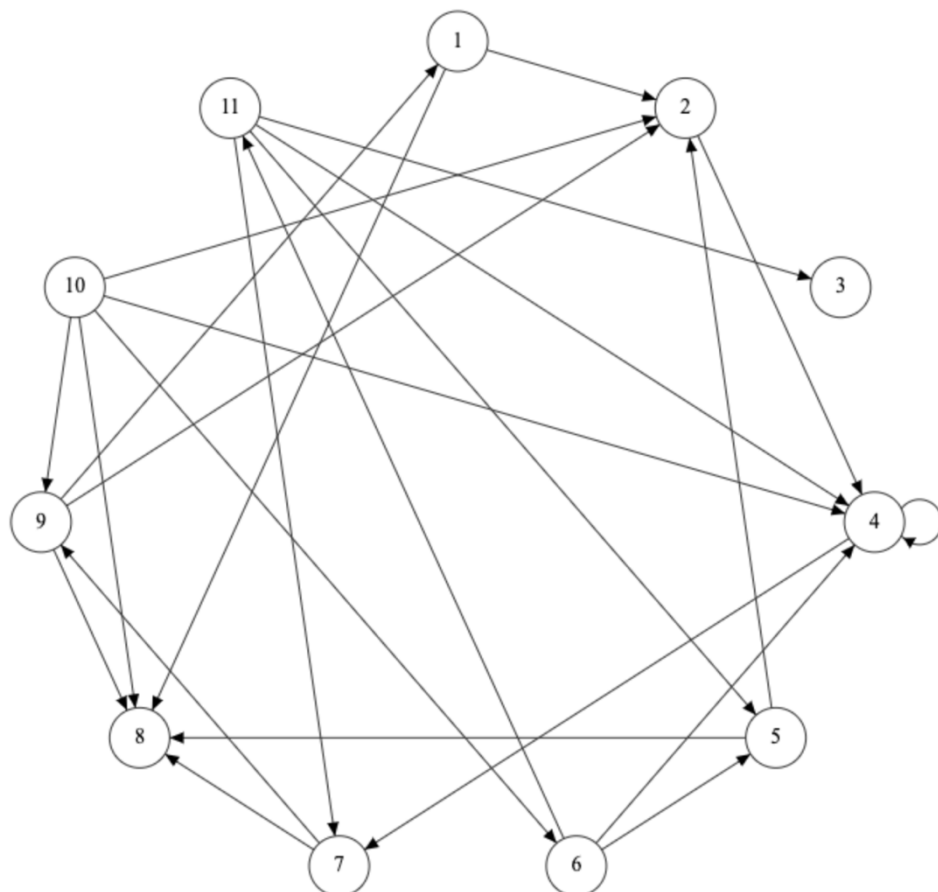
0	1	0	0	0	0	0	0	1	1	0	0
1	0	0	1	1	0	0	0	0	1	1	0
0	0	0	0	0	0	0	0	0	0	0	1
0	1	0	1	0	1	1	0	0	1	1	0
0	1	0	0	0	1	0	1	0	0	0	1
0	0	0	1	1	0	0	0	0	1	1	0
0	0	0	1	0	0	0	0	1	1	0	1
1	0	0	0	1	0	1	0	1	1	0	0
1	1	0	0	0	0	1	1	0	1	0	0
0	1	0	1	0	1	0	1	1	0	0	0
0	0	1	1	1	1	1	0	0	0	0	0

Ненапрявлена матриця суміжності

Variant 4111 🎀

Directed

Undirected

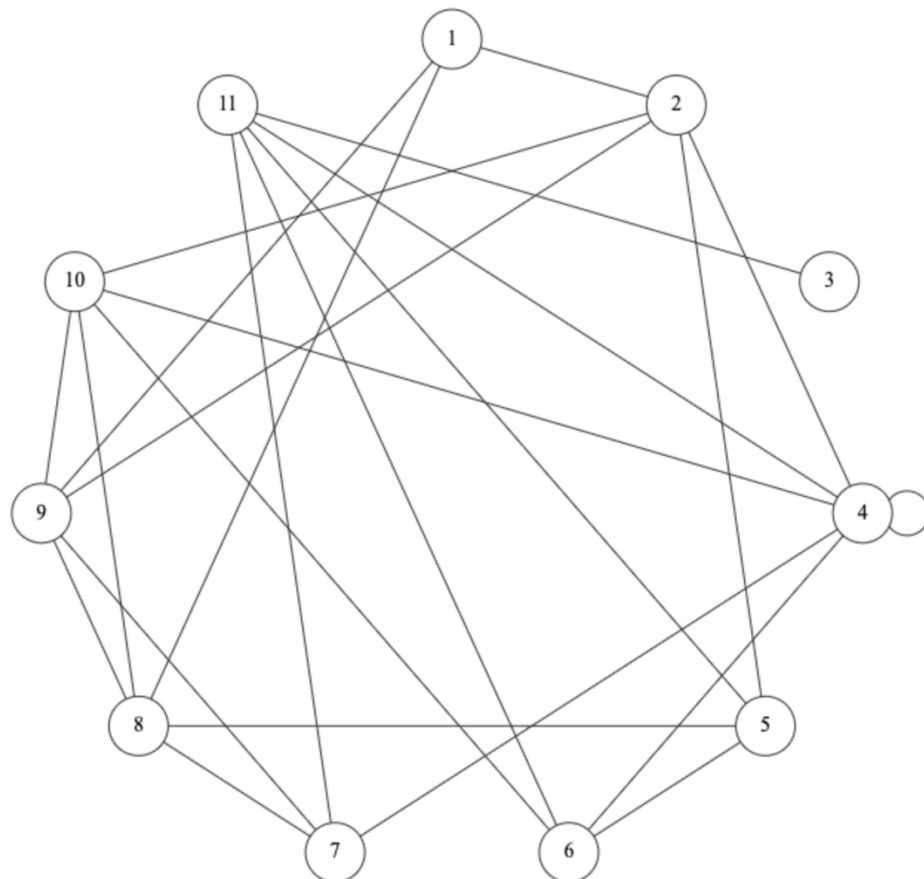


Направлений граф

Variant 4111

Directed

Undirected



Ненаправлений граф

Висновок:

Були набуті навички роботи з `<canvas>` та різними математичними алгоритмами. Оскільки в JavaScript немає функції `srand(seed)`, я дослідила це питання та реалізувала власний псевдовипадковий генератор за алгоритмом Парка–Міллера з метою побудови матриці, аби в результаті отримати послідовність чисел $\in(0;1)$ для побудови матриці. Далі було створено напрямлену матрицю `dirMatrix`, де кожен елемент заповнюється значенням 1 або 0 в залежності від того, чи перевищує випадкове число, помножене на коефіцієнт k , поріг 1. Для того, щоб уникнути двонаправлених ребер (тобто коли одночасно є стрілки $i \rightarrow j$ і $j \rightarrow i$), одна з них випадково видаляється. Ненаправлена матриця `undirMatrix` будується з напрямленої, просто віддзеркалюючи зв'язки — якщо між двома вершинами є хоча б одна стрілка, то зв'язок вважається ненаправленим. Після створення матриць визначаються координати всіх вершин, які рівномірно розташовуються по колу. Для малювання графа використовується функція `drawGraph`, яка спочатку малює ребра, перевіряючи, чи вони повинні бути прямими чи вигнутими (для уникнення перетинів із іншими вершинами). Для цього використовується допоміжна функція `distanceToLine`, яка обчислює відстань від точки до відрізка. Ребра малюються або як прості лінії, або як квадратичні криві, а якщо граф напрямлений — додаються стрілки. Особливий випадок — петлі (ребра, що ведуть у ту ж вершину), вони малюються окремо функцією `drawSelfLoop`, яка враховує розташування вершини та малює дугу з правильно орієнтованою стрілкою. Для користувача передбачені кнопки, які перемикають режим відображення: або малювати напрямлений граф, або ненаправлений, а також виводити відповідну матрицю суміжності у консоль. Отже, під час виконання лабораторної роботи я набула практичних навичок представлення графів у комп'ютері та ознайомилась з принципами роботи ОС.